FULL STACK LAB

LAB SHEET 12 : MINI FOOD DELIVERY APP (QUICKBITE)

Name: Shreya Choudhary

Course: B.Tech CSE – 3rd Year

Semester: 5th Semester

Roll No.: 61

Section: A

Course Name: Full Stack

Course Code: BTCS303P

Date: 14/10/2026

Problem Statement

Build “QuickBite” — a mini food delivery web application. Customers can browse menu items, add items to a cart and place orders. Restaurant admin can add, edit or delete menu items and update order status until delivery.

1. Repository Setup and Architecture

Create QuickBite/ with server.js, public/index.html, public/styles.css, public/app.js, data/db.json, package.json and README.md. Architecture: Browser → responsive HTML/CSS/JavaScript → Express REST API → JSON-file database. The frontend calls the backend through fetch; menu items and orders persist after refresh and server restart.

2. Database Design

```
// MenuItem
{ id, name: String, price: Number, category: String,
  isAvailable: Boolean }

// Order
{ id, customerName, phone, address,
  items: [{ itemId, name, price, qty }],
  totalAmount, status, createdAt }

// Valid statuses
Placed -> Preparing -> Out for Delivery -> Delivered
Placed -> Cancelled
```

Menu item name/category are required and price must be greater than zero. Phone must contain 10 digits. Orders need at least one item and every quantity must be an integer of one or more. The server calculates totalAmount from current menu prices; it never trusts a client-supplied total.

3. REST API Endpoints

Method	Endpoint	Purpose
GET	/api/items?search=&category=	List/filter menu
POST	/api/items	Add menu item
PUT	/api/items/:id	Edit menu item
DELETE	/api/items/:id	Delete menu item
POST	/api/orders	Place order
GET	/api/orders	List newest orders
PATCH	/api/orders/:id/status	Advance status/cancel

4. Back-End Implementation (server.js)

The Express server validates input, persists data in data/db.json, recalculates order totals from menu prices, rejects unavailable items and enforces the status transition rules.

```
app.get("/api/items", (req, res) => {
  const db = readDB();
  let items = db.items;
  const q = String(req.query.search || "").toLowerCase();
  const cat = String(req.query.category || "");
  if (q) items = items.filter(i => i.name.toLowerCase().includes(q));
  if (cat) items = items.filter(i => i.category === cat);
  res.json(items);
});

app.post("/api/orders", (req, res) => {
  const { customerName, phone, address, items } = req.body || {};
  if (!customerName || !/^\d{10}$/.test(String(phone)) || !address)
    return res.status(400).json({error:"Check name, phone and address"});
  if (!Array.isArray(items) || !items.length)
    return res.status(400).json({error:"Cart is empty"});
  const db = readDB(), lines = [];
  for (const line of items) {
    const item = db.items.find(i => i.id === line.itemId);
    const qty = Number(line.qty);
    if (!item || !item.isAvailable || !Number.isInteger(qty) || qty < 1)
      return res.status(400).json({error:"Invalid or unavailable item"});
    lines.push({itemId:item.id, name:item.name, price:item.price, qty});
  }
  const totalAmount = lines.reduce((s,i)=>s+i.price*i.qty,0);
  const order = {id:crypto.randomUUID(), customerName, phone, address,
    items:lines, totalAmount, status:"Placed", createdAt:new Date().toISOString()};
  db.orders.push(order); writeDB(db);
  res.status(201).json(order);
});
```

5. Front End (public/app.js)

```
async function api(url, options={}) {
  const res = await fetch(url, {
    headers: {"Content-Type":"application/json"},
    ...options
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || "Request failed");
  return data;
}

async function loadMenu() {
  const q = new URLSearchParams({
    search: document.querySelector("#search").value,
    category: document.querySelector("#category").value
  });
  const items = await api("/api/items?" + q);
  renderMenu(items);
}

// Checkout submits item IDs and quantities only.
// Server fetches prices and calculates the final total.
async function placeOrder(customerName, phone, address, cart) {
  return api("/api/orders", {method:"POST", body:JSON.stringify({
    customerName, phone, address,
    items:cart.map(x=>({itemId:x.id, qty:x.qty}))
  })});
}
```

6. Testing and Validation

Verify that menu search and category filters work; adding/removing cart quantities updates the total; invalid phone numbers and empty carts are rejected; unavailable or missing items cannot be ordered; and the server calculates the correct total. Test order status progression, cancellation only while status is Placed, missing item update/delete returning 404, and malformed input returning 400.

7. Run and Deliver

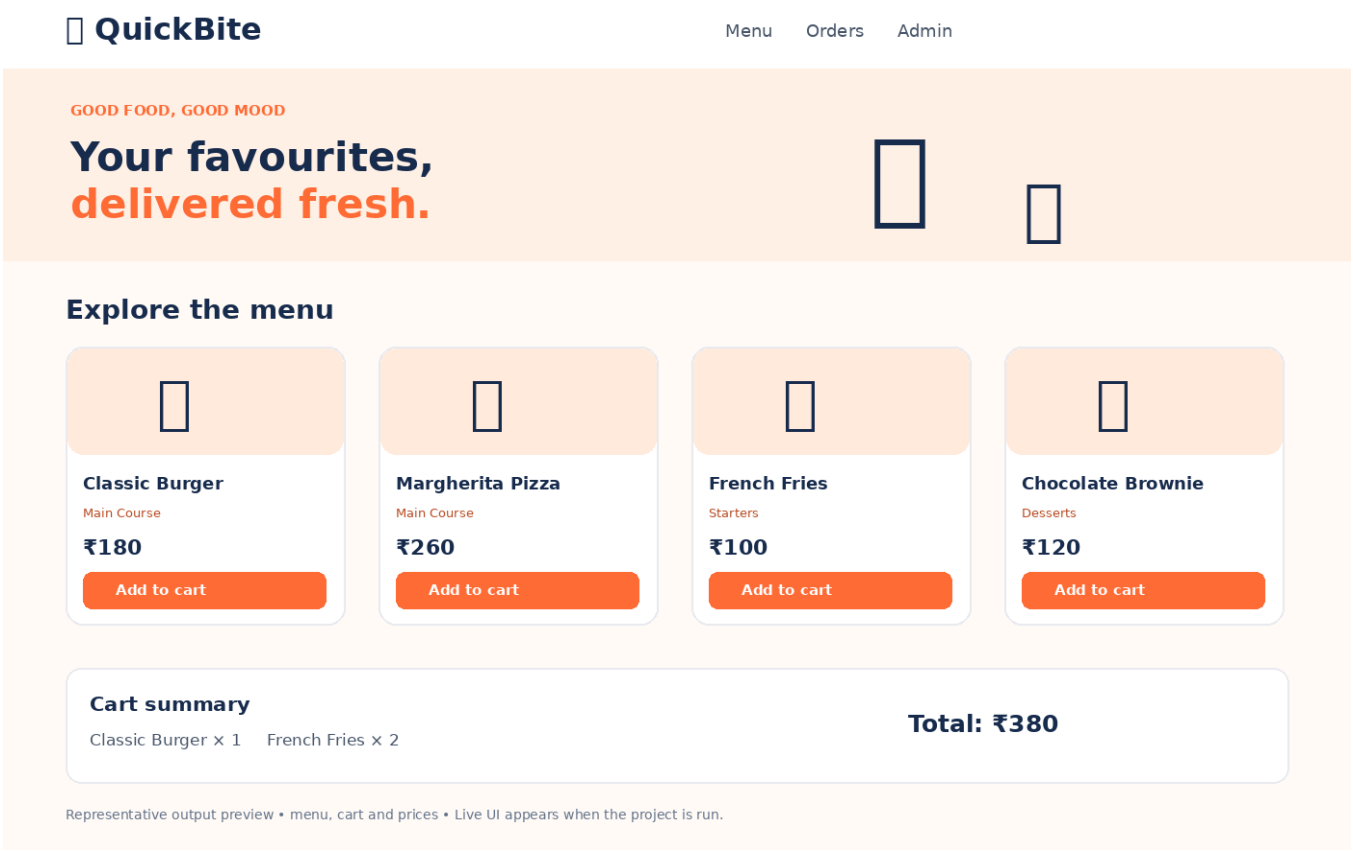
```
npm install
npm start
# Open http://localhost:5000

# Example API checks
GET    /api/items
GET    /api/items?search=burger
POST   /api/orders
GET    /api/orders
PATCH  /api/orders/:id/status
```

The project includes README setup instructions, API list, responsive menu/cart/checkout/orders/admin views, JSON persistence, and a source ZIP. Admin endpoints are exposed for this lab demo; a production version should add authentication and authorization.

Output:

QuickBite — Menu and Cart Interface



The preview shows the menu cards, prices, Add to Cart controls and cart total. Run the project to verify live checkout, order status and admin features. This is a visual mock-up, not a screenshot from a running browser.